

IA-32 架构和汇编编程

School of Automation Science and
Electrical Engineering
Dr. Prof. Ni LI

内容

- IA-32 (X86-32) 介绍
- 80386
 - CPU 架构
 - 工作方式
 - 寄存器
 - 寻址方式
 - 内存管理
- 32-bit 汇编编程

IA-32 (Intel Architecture)

- CPU 设计思想

面向单任务

--->

面向多任务

需要更多内存
内存充分利用

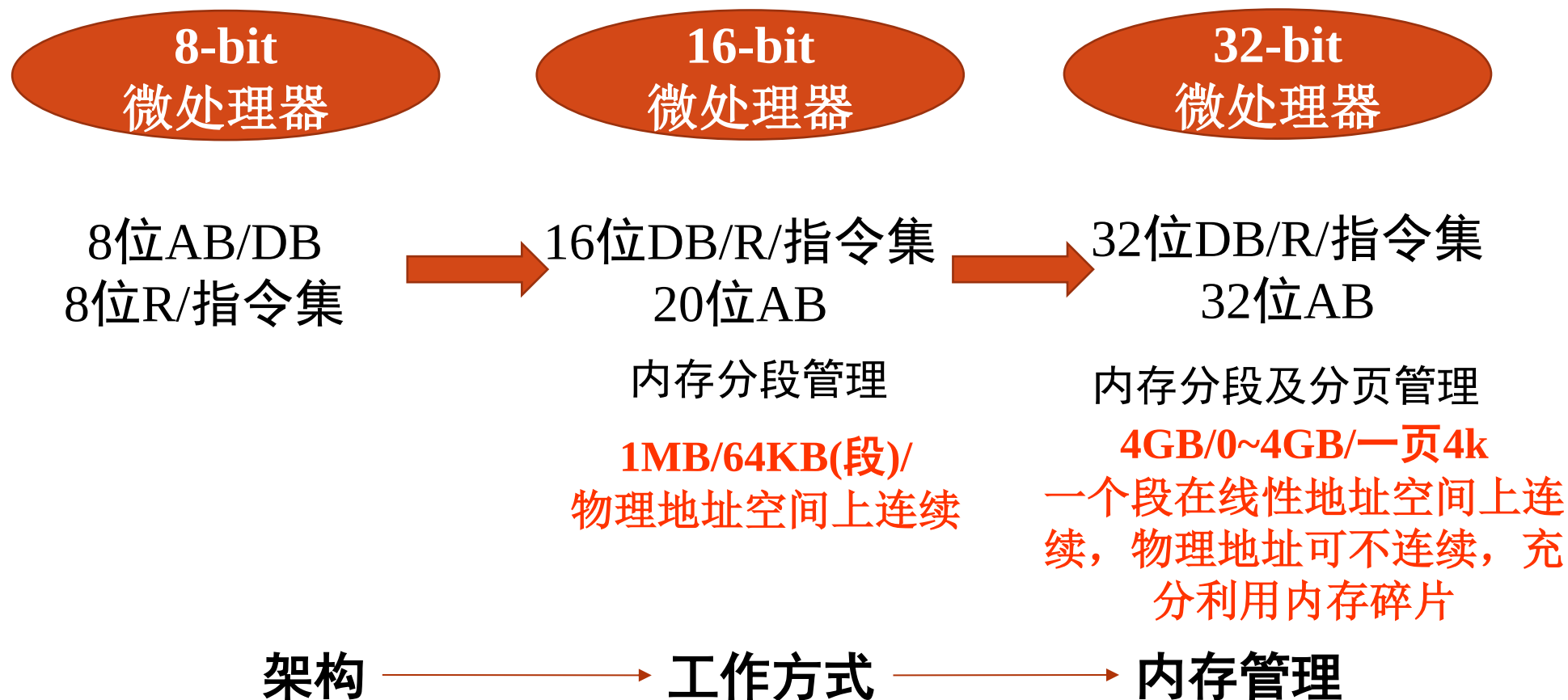


分页管理: 1页为4k, 提高内存存取数据效率, 有效利用内存碎片

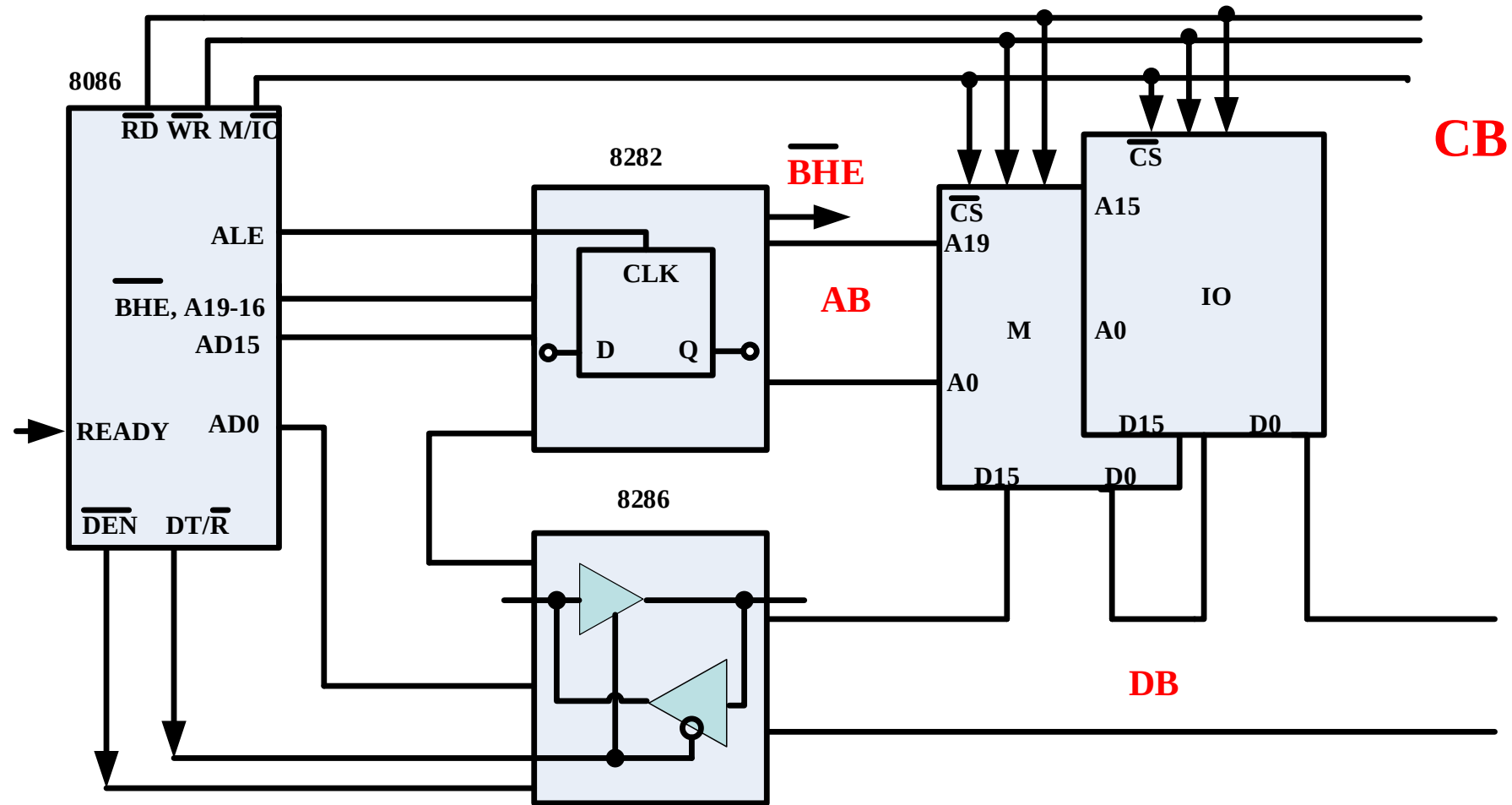
虚拟存储器: 将磁盘映射到内存, 线性地址空间超过实际物理地址空间

IA-32 (X86-32)

- X86-16: 8086
- X86-32: IA(Intel Architecture)-32



8086 系统配置

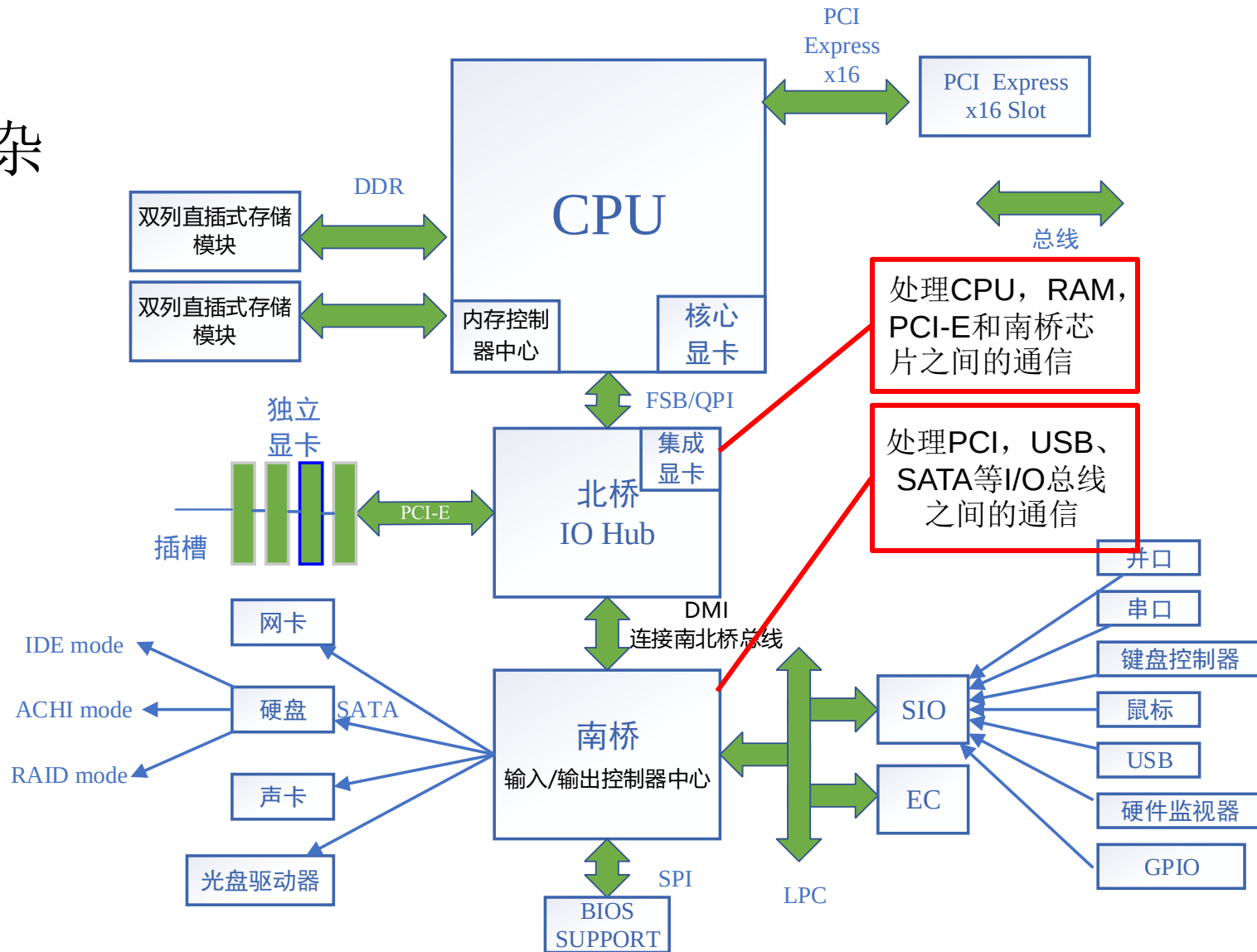


8282 / 74LS373

8286 / 74LS245

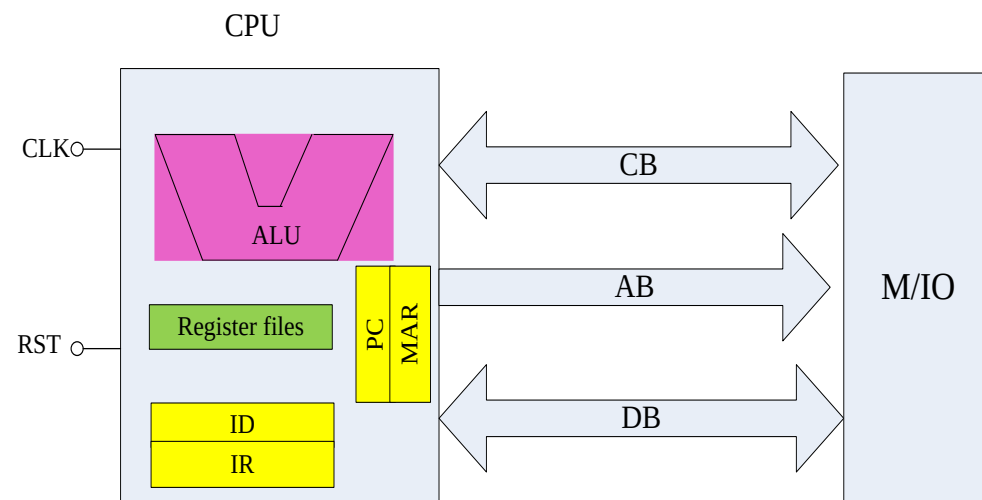
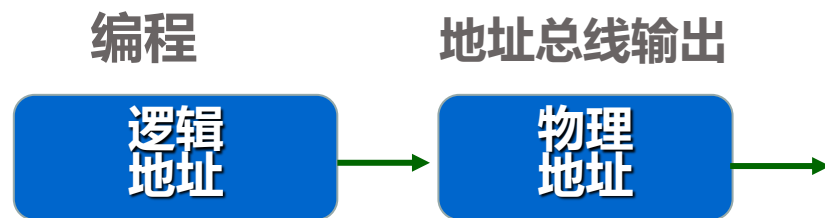
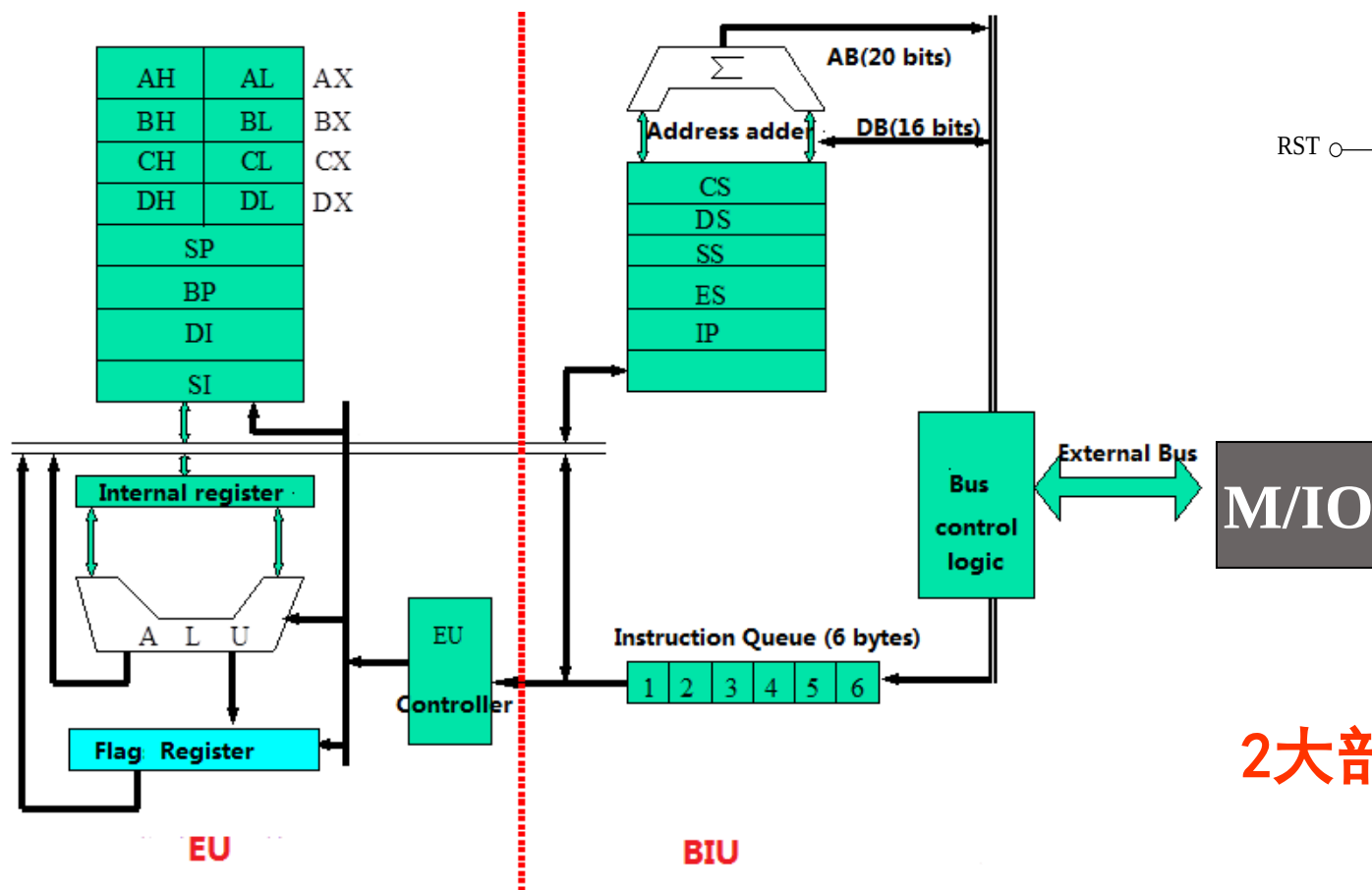
X86 系统架构

- 外设多
- 总线结构复杂



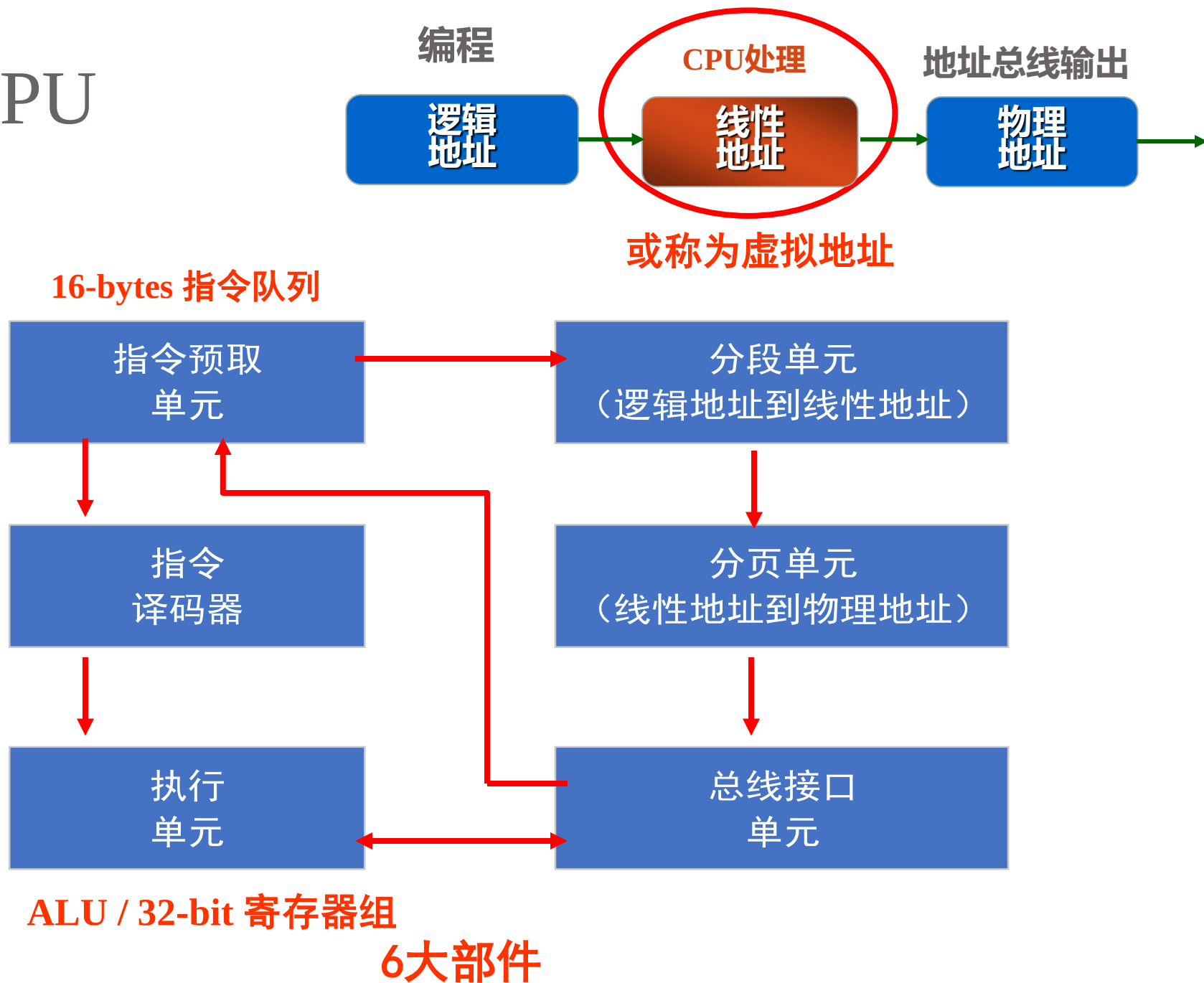
CPU 系统架构

- 8086 系统架构
 - 取指令, 译码和执行



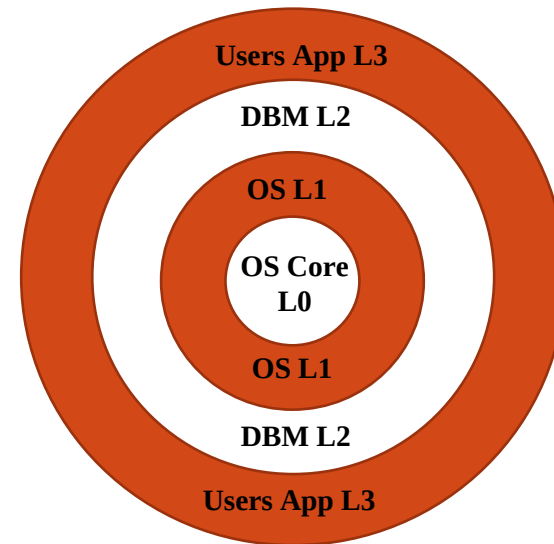
2大部件

80386 CPU



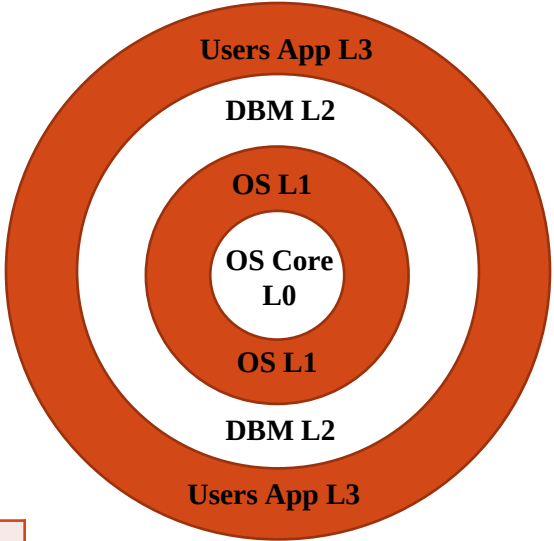
工作方式

- 4 个权限等级: 使用CPU资源的权限
 - 0-3 level
- 0-level: OS Core (使用整个CPU权限)
 - 操作系统的固定程序: 内存管理, 访问控制
- 1-level: OS
 - 操作系统中可变的核程序: 设备驱动器等
- 2-level: 数据库管理/办公自动化软件
- 3-level: 用户程序



工作方式

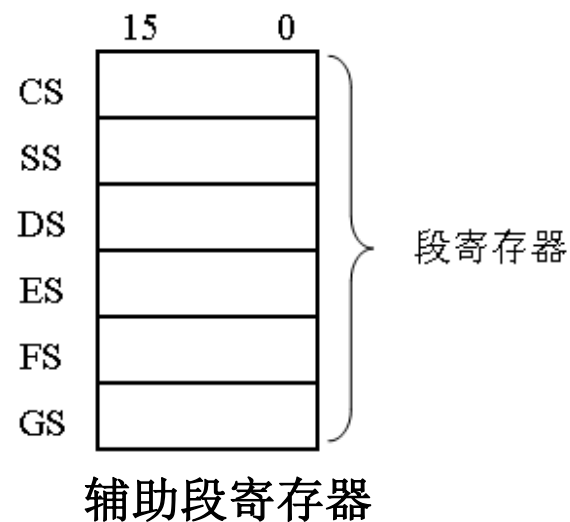
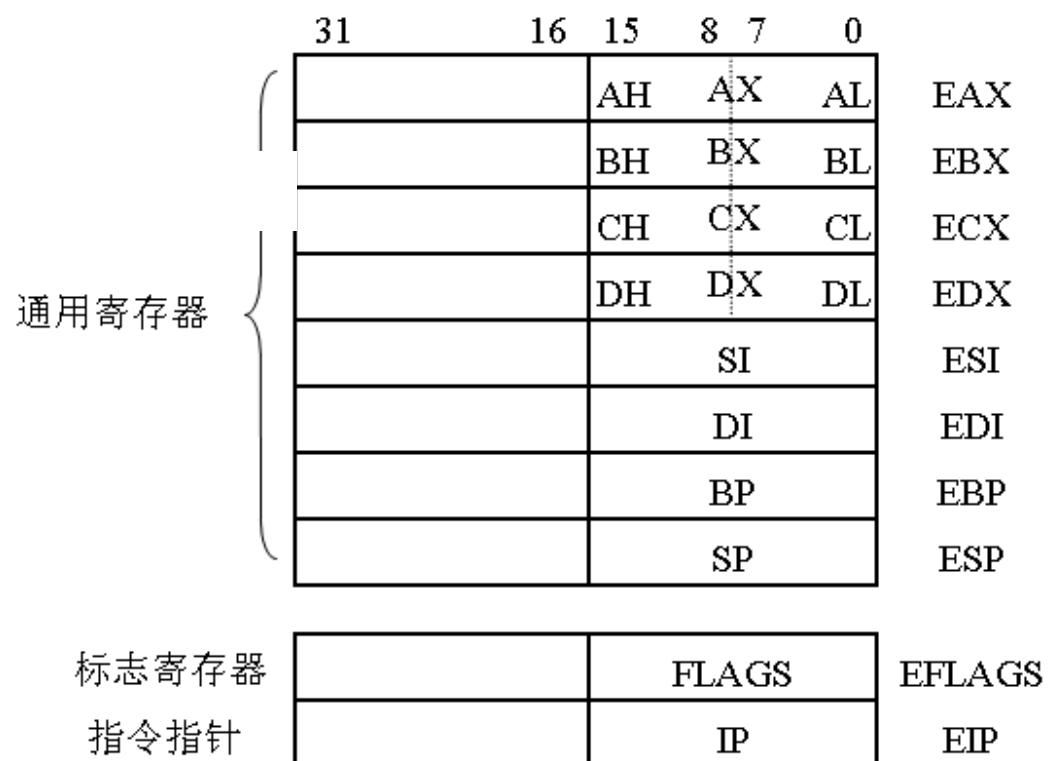
- 内存寻址，任务，权限等级



工作模式	内存寻址空间	任务	权限等级
实地址模式	1 MB	单任务	Privilege 0
保护模式	分段及分页管理，直接寻址4GB 虚拟内存（大于实际物理内存， 对应虚拟地址）	多任务	Privilege 0~3
虚拟 8086 模式 （MS-DOS）	1 MB (每个任务)	多任务	Privilege 3
系统管理模式	4GB 无分段与分页管理	不由操作系统控制 （硬件启动）	Privilege 0



IA-32 寄存器



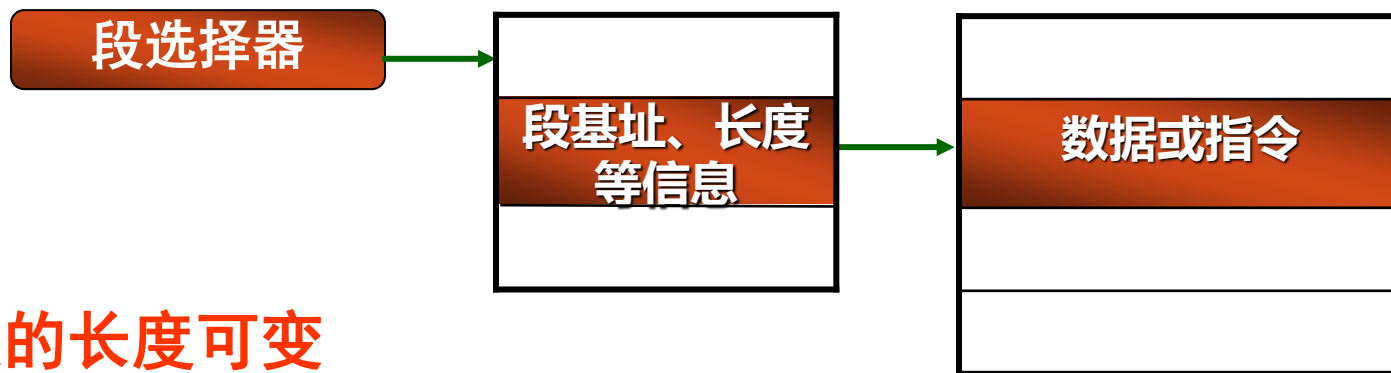
段寄存器

- 段寄存器(16-bit) 与实模式兼容
 - 段选择器 (16-bit)-> (64-bit)段描述符

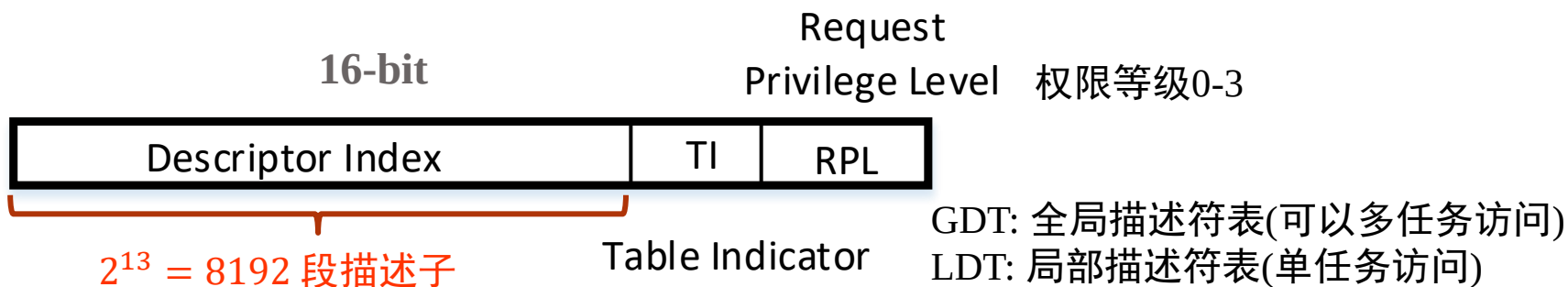
16-bit 段选择器
与实模式/V86 模式兼容

64-bit 段描述子

内存



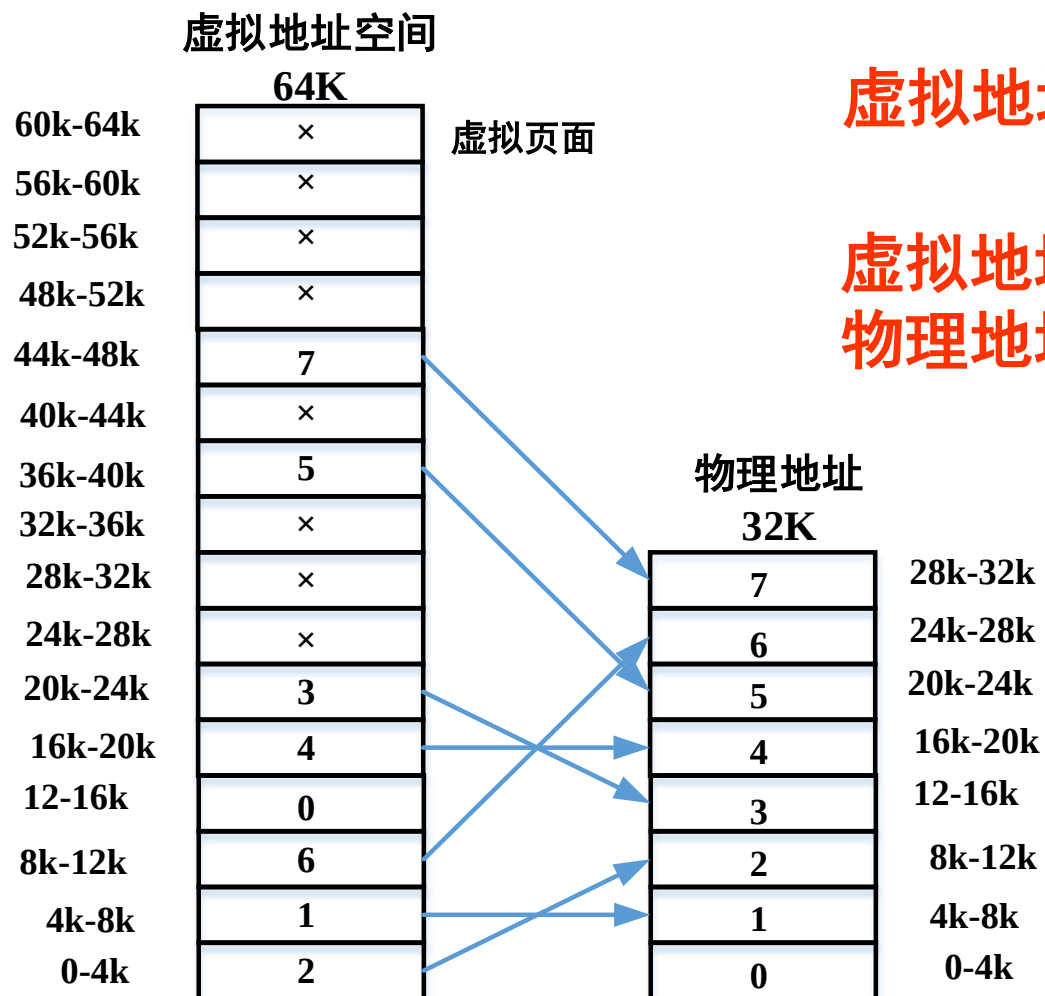
段的长度可变



内存管理---保护模式

- 段描述符
 - 可变长度→ (0~4GB) 分为不同的页存放
 - 段属性→赋予属性和保护信息，通过4级保护，实现程序间，用户和系统间的隔离和保护，有效防止多任务环境下各模块对存储器的越权访问
- 分页管理
 - 4KB→磁盘簇
 - 实现内存和硬盘间的数据映射
 - 实现连续的线性地址空间与不连续的存储器物理地址直接的映射（一个程序可以分不同的物理上不连续的页存放）
 - 有效使用存储器碎片

内存管理---保护模式



虚拟地址空间 > 内存物理地址

虚拟地址空间连续（线性地址），
物理地址不连续

线性地址映射到物理RAM或者磁盘上的交换空间

寻址方式

- 8086存储器寻址
 - 直接寻址
 - 寄存器间接寻址
 - $EA = (EAX, EBX, ECX, EDX, ESP, EBP, ESI, EDI)$
 - 寄存器相对寻址
 - $EA = (EAX, EBX, ECX, EDX, ESP, EBP, ESI, EDI) + \text{偏移量}$
 - 基址加变址寻址
 - $EA = (EAX, EBX, ECX, EDX, ESP, EBP, ESI, EDI) + (EAX, EBX, ECX, EDX, EBP, ESI, EDI)$
 - 相对基址加变址寻址
 - $EA = (EAX, EBX, ECX, EDX, ESP, EBP, ESI, EDI) + (EAX, EBX, ECX, EDX, EBP, ESI, EDI) + \text{偏移}$
- $EA = \text{基址}R + \text{变址}R + \text{偏移}$

寻址方式

- 80386寻址方式

- $EA = \text{基址}R + \text{变址}R * \text{Scale} + \text{偏移量}$
- Scale: 1,2,4,8

- 新寻址方式

- 带比例的变址寻址 访问一个数组

- $EA = (EAX, EBX, ECX, EDX, EBP, ESI, EDI) * \text{scale}$
- 一维数组操作（数组元素可以是2/4/8 bytes）
- Eg. `MOV ECX, [ESI*4]` ESI=0、1、2.....

- 带比例的相对变址寻址

- $EA = (EAX, EBX, ECX, EDX, EBP, ESI, EDI) * \text{scale} + \text{偏移量}$
- 一维数组操作（数组元素可以是2/4/8 bytes）
- Eg. `MOV ECX, [ESI*4+80H]` ESI=0、1、2.....

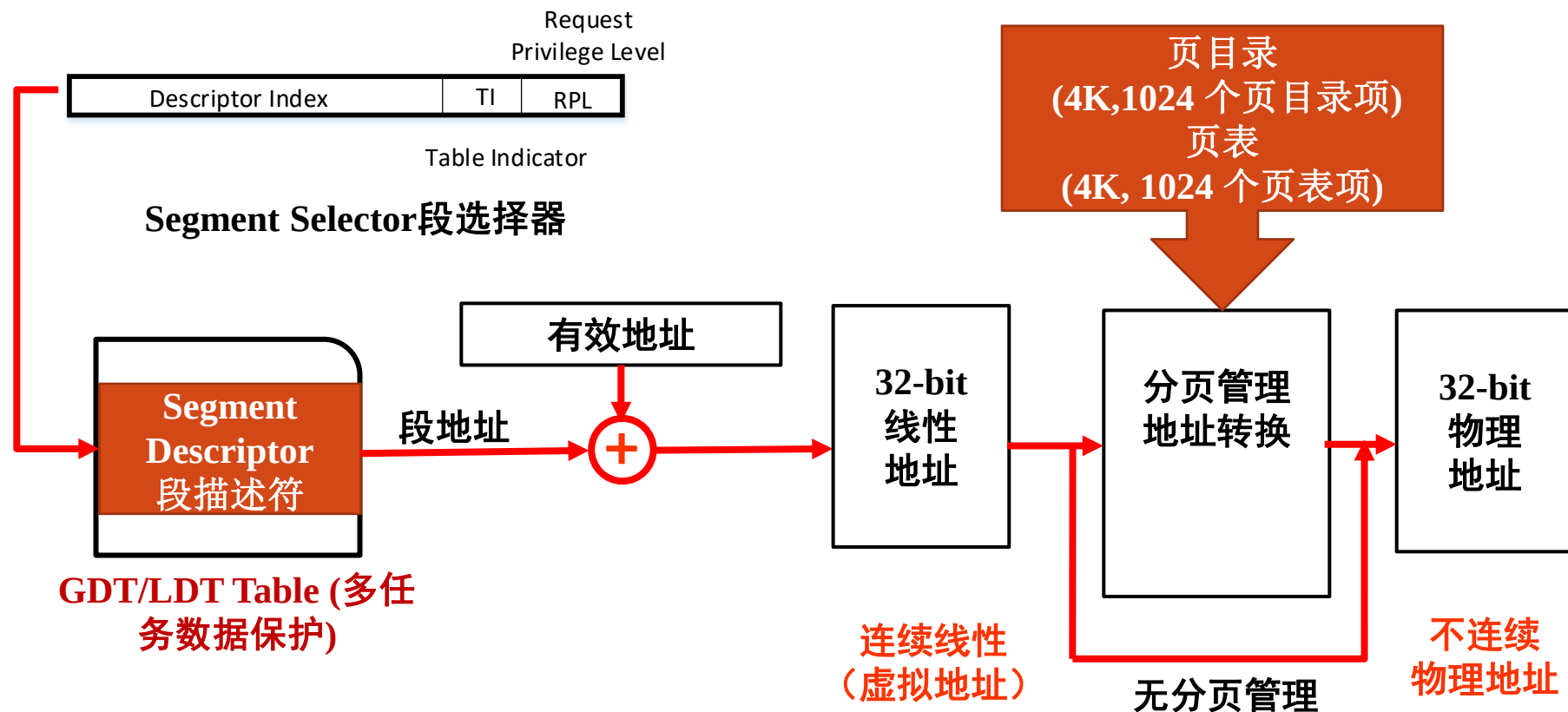
便于访问不同数据类型数组

寻址方式

比例因子

- 80386寻址方式
 - $EA = \text{基址}R + \text{变址}R * \text{Scale} + \text{偏移量}$
 - Scale: 1,2,4,8
- 新寻址方式
 - 带比例的基址加变址寻址
 - $EA = (\text{EAX}, \text{EBX}, \text{ECX}, \text{EDX}, \text{ESP}, \text{EBP}, \text{ESI}, \text{EDI}) + (\text{EAX}, \text{EBX}, \text{ECX}, \text{EDX}, \text{EBP}, \text{ESI}, \text{EDI}) * \text{scale}$
 - Eg. MOV ECX, [EBX+ESI*4]
 - 带比例相对基址加变址寻址
 - $EA = (\text{EAX}, \text{EBX}, \text{ECX}, \text{EDX}, \text{ESP}, \text{EBP}, \text{ESI}, \text{EDI}) + (\text{EAX}, \text{EBX}, \text{ECX}, \text{EDX}, \text{EBP}, \text{ESI}, \text{EDI}) * \text{scale} + \text{偏移量}$
 - Eg. MOV ECX, [EBX+ESI*4 + 4]

地址生成



- 80386地址

- 有效地址EA
- 逻辑地址->线性地址
- 线性地址->物理地址

页太小，内存页数太多，管理内存页数组比较大；页太大，一个进程拥有的内存是页的整数倍，分配空间时会导致碎片

汇编编程

.data ; 数据段定义

Msg byte 'Hello, Assembly!', 0ah, 0dh, 0

.code ; 代码段定义

main proc

start:

mov eax, offset msg

.....

exit 0 ;

main endp

end start(main)

IO 处理

- 16-bit 编程
 - DOS子程序（INT 21H）
- 32-bit 编程
 - 使用C程序： printf
 - 混合编程： C语言和汇编语言
 - Visual Studio (Configuration->platform Win32)

冒泡法——16-bit program

.....

ADDR2:

MOV CX, BX ; 外循环与内循环次数相同

MOV SI, OFFSET STR

NEXT:

MOV AL, [SI]

CMP AL, [SI+1]

JBE NEXT1; 小于则继续比较

XCHG AL,[SI+1];大于则交换

MOV [SI], AL

NEXT1:

INC SI; 地址加1

LOOP NEXT

DEC BX

JNZ ADDR2

.....

冒泡法—32-bit program

```
#include<stdio.h>
#include<stdlib.h>
int nums[]={12,36,81,3,75,42}; //numbers to be sorted

int main()
{
    int i=0;
    printf("before:\n");
    for (i = 0;i < 6;i++)
    {
        printf("%d",nums[i]);
        printf(" ");
    }
    bubble(&nums[0],6); //调用汇编程序
    printf("\n after:\n");
    for (i = 0;i < 6;i++)
    {
        printf(" %d", nums[i]);
        printf(" ");
    }
    printf("\n");
    system("pause");
    return (0);
}
```

Input

Output

冒泡法—32-bit program

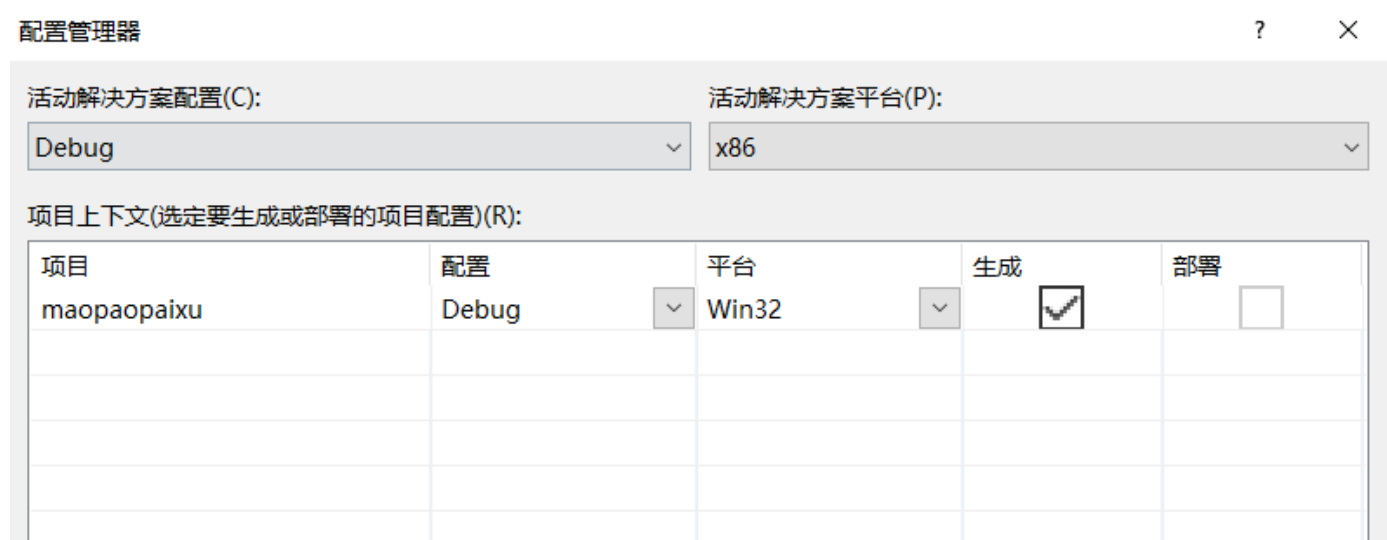
- int 整型数据----4 Bytes
- 寻址方式
 - 访问数组
 - 带比例的基址加变址寻址方式
 - $[ESI+EDX*4]$ $EDX=0,1,2\dots$
 - 带比例的相对基址加变址寻址方式
 - $[ESI+EDX*4+4]$ $EDX=0,1,2\dots$

冒泡法—32-bit program

```
void _stdcall bubble(int *nums, int count) //定义bubble函数
{
    __asm {
        MOV ESI,[nums]           //在c++语言中插入汇编语言程序
                                   //ESI存储数组nums的起始地址
        MOV EBX,count            //输入数据个数
        DEC EBX                  //EBX外循环次数, ECX内循环次数, n-1次外循环和内循环
        outerloop:
            MOV ECX,EBX           //内循环次数
            MOV EDX,0             //EDX自增, 用于寻址数组不同元素
            innerloop:
                MOV EAX,[ESI+EDX*4] //将上边的数放入EAX
                CMP EAX,[ESI+EDX*4+4] //EAX与下边的数比较大小
                JBE nextstep        //EAX小或者相等不进行任何操作
                XCHG EAX,[ESI+EDX*4+4] //交换数据
                MOV [ESI+EDX*4],EAX
            nextstep:
                INC EDX
                LOOP innerloop
                DEC EBX
                JNZ outerloop
    }
}
```

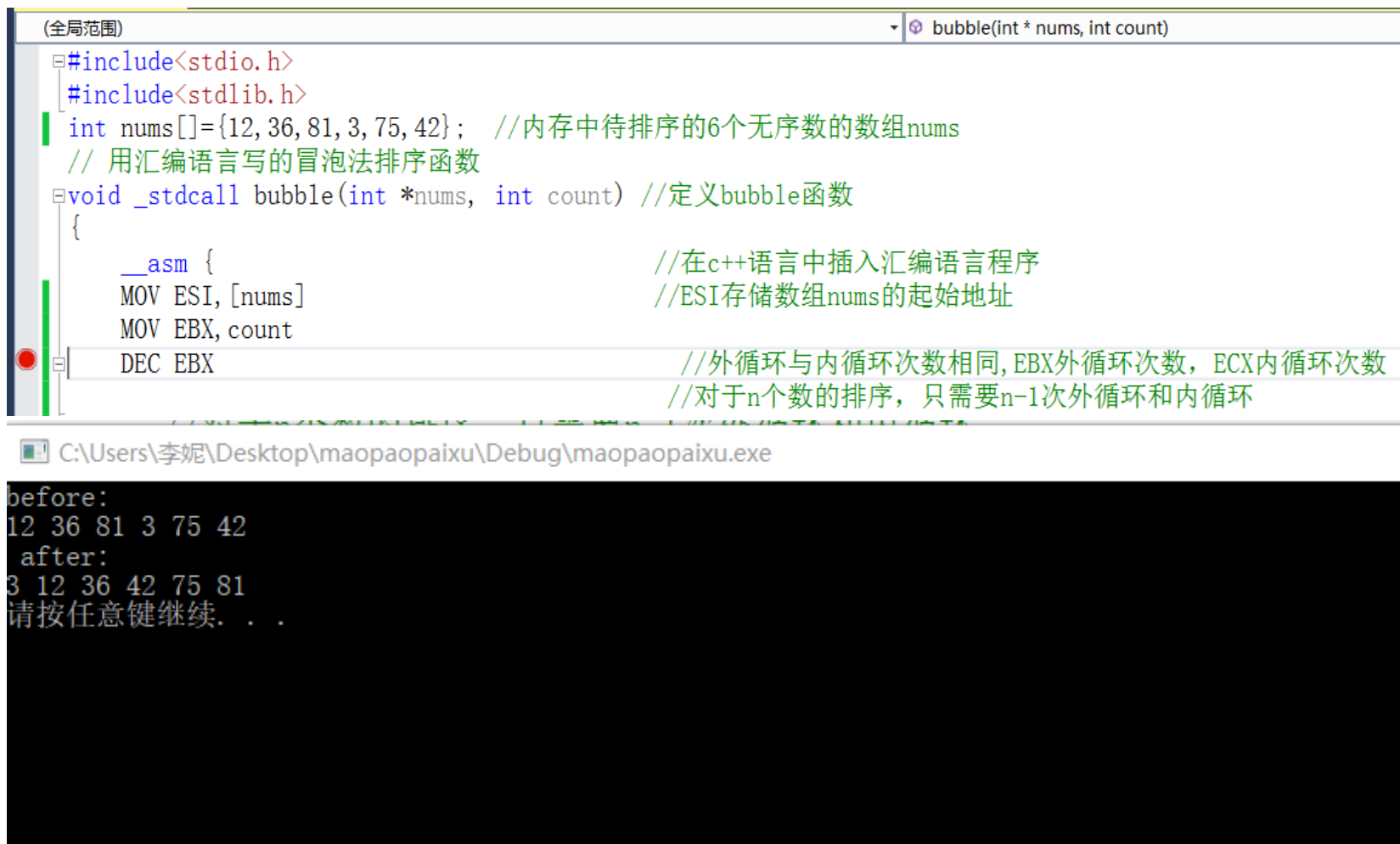

Visual Studio

- Configuration



Visual Studio

- Debug and Result



The screenshot displays the Visual Studio IDE. The top toolbar shows the 'Debug' button (a magnifying glass over a play icon) and the 'Run' button (a play icon). The 'bubble(int * nums, int count)' function is selected in the toolbar. The code editor shows the following C++ code:

```
(全局范围) bubble(int * nums, int count)
#include<stdio.h>
#include<stdlib.h>
int nums[]={12, 36, 81, 3, 75, 42}; //内存中待排序的6个无序数的数组nums
// 用汇编语言写的冒泡法排序函数
void _stdcall bubble(int *nums, int count) //定义bubble函数
{
    __asm { //在c++语言中插入汇编语言程序
        MOV ESI, [nums] //ESI存储数组nums的起始地址
        MOV EBX, count
        DEC EBX //外循环与内循环次数相同,EBX外循环次数,ECX内循环次数
        //对于n个数的排序,只需要n-1次外循环和内循环
```

The bottom of the image shows the output window with the following text:

```
before:
12 36 81 3 75 42
after:
3 12 36 42 75 81
请按任意键继续. . .
```

大作业3

- 冒泡法排序
- 选作内容：
 - 输入一个数计算阶乘并输出显示